打造穩定的 AI 開發工作環境

AI Coding Agent 實戰工作坊 — 中階場 (Claude)

目標： 了解 Prompt、Context、Harness 如何配合，讓 AI 在團隊裡更穩定地工作

John Wu | Apr 2026

今天的旅程

時段	做什麼
觀念講解	用餐廳比喻看 Prompt / Context / Harness
Live Demo	看同一個需求如何一步步變穩
練習時間	套到你自己的工作場景
回顧 Q&A	重點整理 + 課後練習說明

> 重點不是多學幾個名詞，而是知道怎麼把 AI 放進可重複的工作流程裡

Part 1: 觀念講解

用「開餐廳」來比喻

你說的話	餐廳比喻	對應概念
「我想吃辣的」	先做一版試吃再調整	Vibe Coding
「宮保雞丁，不要花生，少油」	講越具體，出餐越準	Prompts
「這是川菜館，大火快炒」	廚房規矩寫好	Project Instructions
食譜 + 檢查點 + 工具清單	標準作業流程	Skill
接上供應商、倉庫、外送系統	對外接口	MCP

今天要升級什麼？

角度	你在做什麼
單次協作	把需求講清楚，讓 AI 幫你完成一次任務
可重複流程	設計規則、流程、工具，讓 AI 能穩定完成很多次任務

> 你不只是想做出一道菜，而是開始設計整間廚房怎麼穩定運作
> 同樣一道菜，有人每次都靠運氣，有人靠制度穩定出餐

今天的工具視角

工具	今天的定位
Claude Code	主要示範工具，展示怎麼設計 AI 工作環境
Codex	偶爾對照，幫你確認概念是跨工具通用的

重點不是比工具，而是看原理怎麼搬

這些概念怎麼搬到你熟悉的工具裡

> 工具名稱會變，但 Prompt / Context / Harness 這套思路很穩

AI 工作環境的三大支柱

支柱	餐廳比喻	白話
Prompt	你怎麼點餐	你給 AI 的指令、限制、輸出要求
Context	菜單、廚房規矩、食材資訊	AI 取得到的專案知識與外部資訊
Harness	廚房制度、權限、安全檢查	AI 可以怎麼做、做到哪裡、何時該停下來

> 中階不是學更多術語，而是知道哪一層沒補好

為什麼同一句需求，結果還是會飄？

常見原因通常不只一個

1. **Prompt 不夠清楚**：任務、限制、輸出格式沒有講明
2. **Context 不夠完整**：AI 不知道 repo 規範、現有 code、外部資料
3. **Harness 不夠穩**：沒有檢查點、權限邊界或 workflow 約束

所以你要學會的不是「寫超長 prompt」

而是判斷：這次要補的是哪一層

> 問題不一定出在你不會問，也可能出在廚房根本不知道規矩

Prompt — 你怎麼點餐

不是每件事都要用同樣鬆緊度

任務類型	點餐方式	建議鬆緊度
資料庫遷移	指定做法、限制風險	**嚴格**
API 實作	講清規格，保留部分空間	**中等**
Code Review	給目標與觀察角度	**中等到寬鬆**
寫文件	講清受眾與格式	**寬鬆**

> 越危險的操作，prompt 越要明確；越探索的任務，可以留多一點空間

Context — 廚房知道什麼

如果 Prompt 是點餐，那 Context 就是：

- 這是一家什麼餐廳
- 廚房有什麼規矩
- 現在有哪些食材
- 有沒有外部系統可以查資料

```
常見 context = 對話內容 + 相關檔案 + 專案指引檔 + 需要時取得的外部資訊
⚠️ context 不只影響品質，也影響成本與效率——塞太多不相關的東西，判斷會變差、資源也會浪費
```

> AI 不只是聽你當下說什麼，還會受它拿得到的資訊影響

Context — 專案指引檔是廚房規矩

概念	Claude Code	Codex	Gemini
專案指引檔	`CLAUDE.md`	`AGENTS.md`	`GEMINI.md`
作用	告訴 AI 專案身份、規範、工作方式	同類型概念	同類型概念
今天定位	主角	對照例子	對照例子

你是在跟 AI 說：「這是我們家的廚房，我們平常就是這樣做事」

> 寫一次，之後每次對話都比較容易站在同一套規則上

Context — 專案指引檔通常寫什麼？

```
# Project Name

## Description
(...)

## Tech Stack
React + Vite, Express, MongoDB

## Coding Conventions
- 元件用 functional component
- API 回傳統一用 { data, error }

## Workflow
- 改 UI 前先確認 RWD breakpoint
- API 改完要跑 integration test
```

不是在列百科全書，而是在定義這間廚房的做事方式

> 寫太少，AI 會亂猜
> 寫太多，真正重要的規則被淹沒

⚠️ ETH Zurich 2026：

AI 自己生的指引檔 → 成功率反而**下降**

人寫但塞太多 → 幾乎沒幫助

> 只寫最小必要規則，效果最好

Context — 專案指引檔怎麼寫才有用？

要寫	不要寫
規則（約束怎麼工作、什麼不能做）	介紹（項目是什麼、目錄結構）
團隊/倉庫獨有的限制	模型本來就會的常識
長期有效的流程	臨時任務（應寫成 Skill 或 Hook）
反面例子「不要這樣做」	正面套話「保持優雅」

> 每一行都佔全局上下文
> 不寫就會出錯的才寫進去，同樣錯誤出現兩次再更新

Context — Prompt 其實是組裝出來的

你寫的不只是一份 CLAUDE.md，而是可組合的模組

```
CLAUDE.md → role → policy → stack → skills → templates
```

每次都載入（核心）　　按需選用

- **核心模組**：身份、規則 — 每次對話都帶著
- **Skills**：怎麼做 — 任務有固定 SOP 時載入
- **Templates**：結構化的資訊格式 — 任務背景、報告、提案都是模板

Harness — 廚房怎麼被管住

機制	餐廳比喻	說明
權限 / approval	哪些事要店長批准	高風險操作先確認
Hooks / 自動檢查	出餐前先過品管	在特定時機自動跑檢查或流程
Workflow 約束	廚房標準流程	先看 code、再改、最後驗證

```
// .claude/settings.json — hook 範例
// 白話：「每次 AI 改完檔案，自動跑一次程式碼檢查」
{ "hooks": { "PostToolUse": [{
    "matcher": "Edit",
    "hooks": [{ "type": "command", "command": "pnpm lint --fix" }]
}]}}
```

> Harness 像安全帶 — 不是綁手綁腳，而是讓你敢放手讓 AI 跑

Harness 的兩個面向

面向	做什麼	餐廳比喻
自動護欄	hooks、權限、檢查點，防止出錯	出餐前品管自動檢查
反覆調校	根據結果回頭修正 Prompts & Skills	每次出餐後檢討，調整食譜和流程

兩者搭配才完整

- 只有護欄，沒有調校 → 規則僵化，遇到新狀況就卡住
- 只有調校，沒有護欄 → 每次都靠人盯，效率上不去

> 執行期護欄與事後調校要分開看，但兩者都會影響整體 workflow 的穩定度

Skills / MCP / Plugins，分別在做什麼？

比喻	概念	白話
點餐方式	Prompt	你怎麼下指令
廚房規矩	CLAUDE.md / AGENTS.md	專案背景與做事方式
標準作業流程	Skills	讓 AI 按步驟完成一種任務
對外接口	MCP	讓 AI 連到外部工具或資料
整套設備包	Plugins	把常用能力打包成可重用組合

> 你是在設計一間會運作的廚房，不是在多買幾個廚具

Skills — 把做事方法變成 SOP

Skill 可以先把常見任務的流程寫好，例如：

- 先收斂需求
- 再拆計畫
- 接著實作
- 最後驗證與收尾

```
模糊需求 -> brainstorming -> plan -> implementation -> verification
```

> Skill 不只是 prompt 模板，而是可重複套用的工作流程

MCP — 讓廚房自己去拿資訊

MCP 可以讓 AI 在需要時自己去查資料或做操作，例如：

- 讀 issue 或 ticket
- 查資料庫結果
- 看團隊討論脈絡
- 讀檔案或文件

關鍵不是「接了多少系統」

而是：**AI 能不能在對的時候，自己拿到它需要的資訊**

> MCP 是對外接口，不是外部系統本身

一句話收斂：中階到底在學什麼？

Prompt

把需求講清楚

Context

讓 AI 知道這間廚房的規矩與資訊來源

Harness

讓 AI 在對的邊界與流程下工作

> 中階不是多裝工具，而是開始設計 **AI 的工作環境**

Part 2: Live Demo

從 0 到 1，讓 AI 按流程做出產品雛形

```
做一個 RWD 的任務管理系統
需要有任務列表、狀態篩選、新增任務表單
桌機與手機都要可用
```

Demo 怎麼對應三大支柱？

- **Prompt / Context**：先定角色與規則
- **Context → Spec**：用 brainstorming 把需求收斂
- **Harness**：用 plan、回饋迴圈推進與修正

Demo 1 — 先建立 Role / Skill 系統

在專案中建立三個 Role Prompts

- **Frontend**：專注 UI 結構、RWD、互動與元件拆分
- **Backend**：專注 API、資料結構、驗證與錯誤處理
- **QA**：專注測試案例、邊界條件與風險提醒

每個角色再配一個 Skill

- **Frontend skill**：先定頁面結構，再定 breakpoint，再拆元件
- **Backend skill**：先定 schema，再定 API，再補 validation
- **QA skill**：先寫 test plan，再補 edge cases 與 E2E checklist

Demo 2 — 建立約束條件

用 `/init` 產生 CLAUDE.md，再把規則寫進去

```
/init 這個專案每次執行任務時，只帶一個角色與對應 Skill，Skill 最多 3 個
```

情境	沒約束	有約束
角色混用	前後端混扮，風格不一致	一次一個角色，職責清楚
Skill 爆量	太多 SOP 互相衝突	最多 3 個，聚焦當前任務
規則遺失	每次對話都要重新交代	寫進 CLAUDE.md，自動生效

Demo 3 — Brainstorm 把需求收斂成 Design Spec

安裝 Superpowers

從一句模糊需求開始

```
/superpowers:brainstorm 我想做一個 RWD 的任務管理系統
```

- 逐步澄清成可落地規格
- 決定架構方案、技術棧

> 關鍵不是直接寫 code，而是把需求先收斂成可以分工與驗收的 spec

Demo 4 — 從 Plan 到逐步實作

先把 spec 變成可執行計畫

- 把 spec 拆出 **N 個 tasks**
- 依序讓 AI 實作每個 task
- 最後用 build / test 當驗收標準

> AI 依照 plan 一步步推進，每完成一步就驗證再繼續

執行順序

1. 後端 schema + API
2. 前端元件 + 頁面
3. QA 補測試案例與驗收

Demo 5 — 根據回饋即時修正

啟動環境驗證

- 請 AI 修改完自動啟動 backend 與 frontend
- 用瀏覽器實際操作，檢查功能是否可用

問題	該補什麼	修正方式
手機版任務卡片排版跑掉	Context（規範）	調整 flex / media query
新增任務後 API 回 404	Prompt（路由細節）	檢查路由與 controller
篩選「已完成」仍顯示全部	Harness（測試檢查點）	確認 query 與 filter

> 成品不會一次到位，關鍵是判斷該補哪一層，再回饋給 AI

Demo 6 — 用 OpenSpec 提出需求變更

1. 安裝 OpenSpec（`/opsx` 是它產生的 slash command 前綴）
2. 專案中啟用 `openspec init`
3. `/opsx:explore`：提出需求
4. `/opsx:propose`：建立需求文件及分工規劃
5. `/opsx:apply`：依照 spec 實作
6. `/opsx:archive`：驗收完成後，把需求歸檔

> OpenSpec 不是文件倉庫，而是需求、設計、實作、驗收的變更帳本

Demo 回顧

重點	要讓觀眾看到什麼
Role & Skill 系統	先定角色，再定 SOP，最後用規則把 AI 約束住
Superpowers	模糊需求可以被逐步收斂成可執行規格
Plan 驅動實作	把 spec 拆成 task，依序實作、逐步驗收
即時回饋修正	判斷該補 Prompt / Context / Harness 哪一層
OpenSpec 工作流	`explore → propose → apply → archive` 讓每次變更都有脈絡
Feedback Loop	用計畫、驗證與回饋迴圈，讓流程越跑越穩

除了 Skill，還有一個好用的層：Templates

模板	用途
`templates/task-context.md`	單次任務背景（最輕量）
`templates/change-context.md`	單一變更（含範圍、影響）
`templates/project-context.md`	專案全貌（最完整）
`templates/code-review.md`	Code Review 報告
`templates/design-proposal.md`	設計提案

> 每個模板都是「一種結構化的資訊格式」

Part 3: 練習時間 (20 min)

把今天的方法套到你的工作環境

> 練習 1 為必做，練習 2 選做，自由實作為課後延伸

練習 1 — 設計你團隊的廚房規矩

你的任務

替你的 repo（或今天的 demo 練習專案）草擬一份 `CLAUDE.md`

至少回答這四題

1. 這是什麼專案？用什麼技術？
2. AI 改 code 前，應先知道哪些規範？
3. AI 做完後，至少要驗證什麼？
4. 有哪些事不能亂做？

> 先寫出 60 分版本，比追求一次寫到 100 分更實際

練習 2 — 替常見任務補上 Harness

想一個你們團隊常做的任務，然後定義權限與檢查點

要回答的問題	範例
哪些操作可以直接做？	讀檔案、跑 lint
哪些操作應該先問？	刪除檔案、改 DB schema
做完之後怎麼驗證？	跑 test suite、檢查 API response

> 很多翻車不是因為 AI 不夠聰明，而是因為沒有安全帶

自由實作（課後延伸）

選一個最接近你工作的題目

> A) 為團隊 repo 草擬一份 `CLAUDE.md`
> B) 把一個模糊需求拆成 Prompt / Context / Harness
> C) 找一個常見任務，設計它的 Skill（從觸發條件到驗收標準）

提醒

做出來的內容最好是你回去就能直接用的

> 不求做很多，先做出一個能落地的版本

Part 4: 回顧

實務提醒：成本與 context 意識

前面提過，context 不只影響品質，也影響成本

這些情況通常都比較耗資源：

行為	常見影響	建議
超長對話	context 越來越重	適時開新對話，保持主題乾淨
一次塞太多不相關資訊	判斷品質下降	只給真正相關的內容
反覆試錯但不收斂	花時間也花額度	退一步先補 context 或規則
不必要的多工具串接	複雜度上升	先解決核心問題，再加工具

> 省資源的方法通常不是少問，而是讓每次提問更有結構

關鍵 Takeaway

1. Prompt + Context + Harness 三者缺一不可

好的結果，通常不是靠單一神 prompt，而是三層一起補

2. `CLAUDE.md` / `AGENTS.md` 是團隊共享的廚房規矩

寫一次，之後每次合作都不用重新對齊規矩

3. Skills + MCP 讓 AI 從聊天工具變成工作環境的一部分

不只會回答，還能按流程做事、在需要時自己拿資訊

> 中階的核心不是「更會用 AI」，而是「更會設計 AI 怎麼工作」

課後練習

練習	怎麼做
寫一版專案指引檔	先列技術棧、規範、驗證流程、禁區
做一次四輪補強實驗	同一任務依序只補 Prompt → Context → Harness → Skill (比較結果差異)
找一個常見流程做 SOP	先用文字寫出步驟，再考慮要不要做成 Skill
結果不穩時怎麼判斷	回頭看是 Prompt / Context / Harness 哪一層沒補好

回去先做一件事

> **在你的 repo 裡，補上最基本的 AI 工作規矩**

下一站：進階場預告

今天你學的是——把 AI 放進穩定的工作環境

進階場會學的是——把多個 Agent 組成可持續運作的流程

中階（今天）	進階（下一場）
餐廳：一間廚房怎麼運作	工廠：多條產線怎麼協調
單一 Agent + 規則	多 Agent 架構 + 自訂 Skill
Prompt / Context / Harness	Agentic Workflow 設計

> 今天打好的基礎（角色、規則、檢查點），進階場會全部用上

Q&A